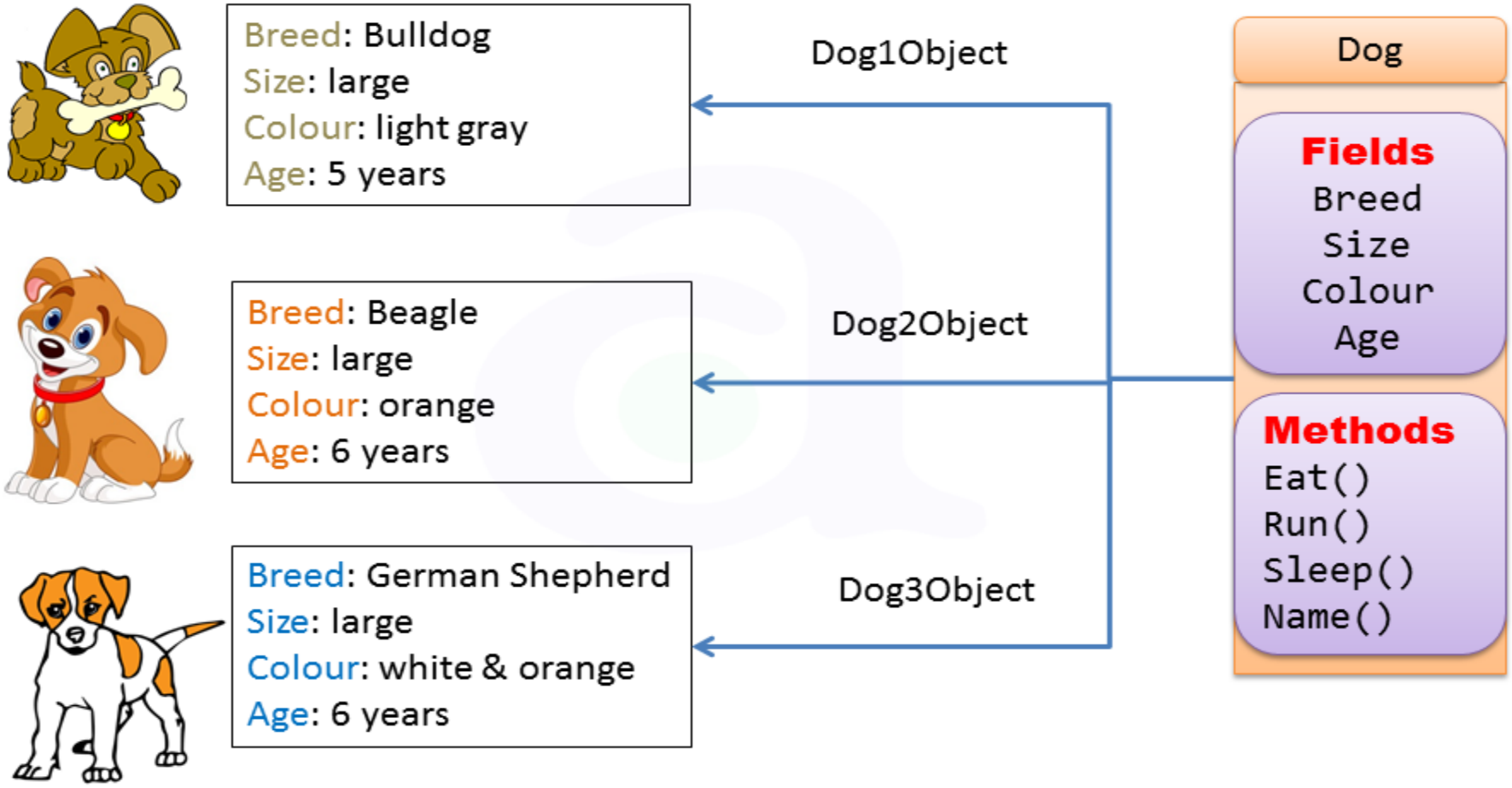
Chapter Three



Java Class and Object

Introduction

- Object-Oriented Programming (OOP) is a programming paradigm which allows you to implement an object-oriented design as a working system.
- Procedural programming is about writing procedures or methods that perform operations on the data, while object-oriented programming is about creating objects that contain both data and methods..
- OOP is organized around the concept of "classes"
- In OOP, a programmer defines a class that describes the properties and behaviors of a particular type of object.
- Objects are created from these classes, and each object has its own set of attributes and methods.
  - Attributes represent the object's data, such as its name, age, or any other relevant information.
  - Methods represent the actions that can be performed on the object, such as modifying its attributes or performing some computation based on its data.
- *One of the key advantages of OOP is that it allows programmers to write code that is modular, reusable, and easy to maintain.*

Defining Java Classes for Objects

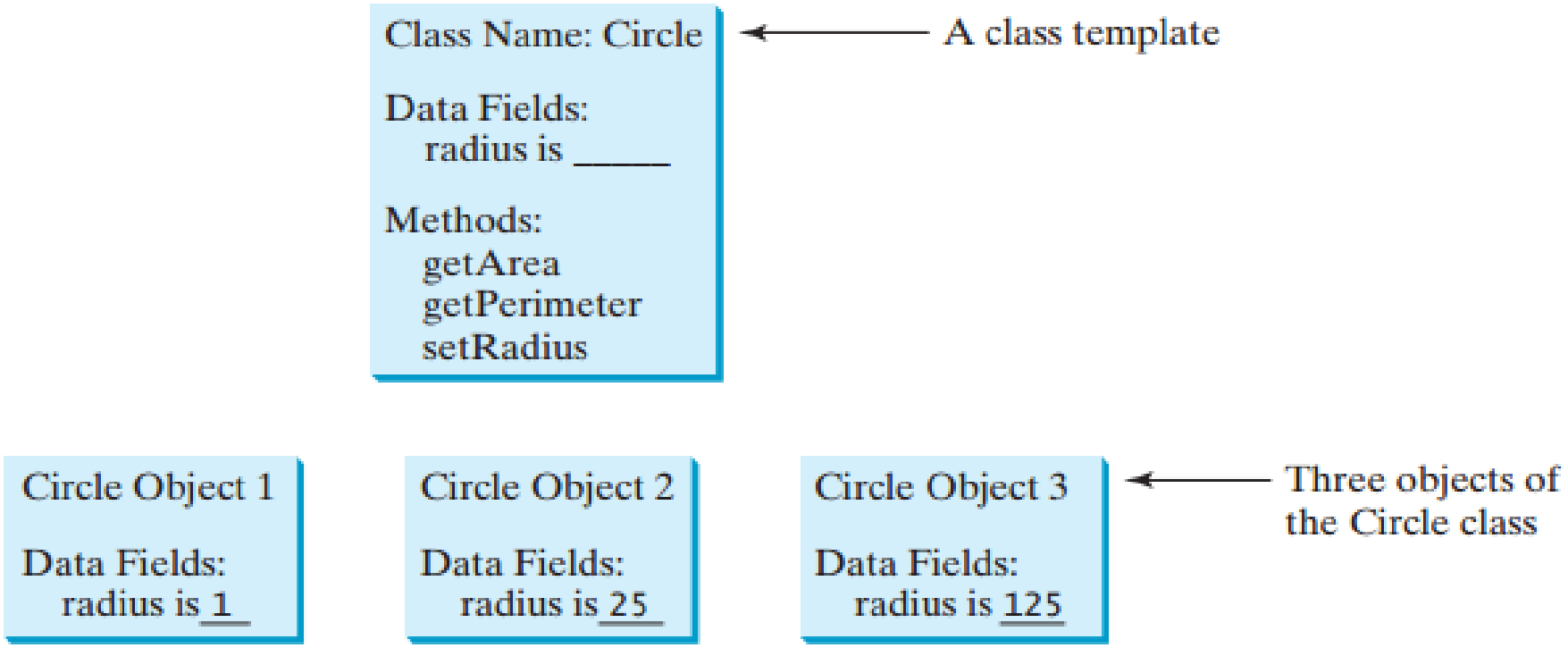
- In Java, a class is a blueprint or a template that defines the properties and behaviors of objects.
- It contains a set of variables, methods, and constructors that define the state and behavior of objects that belong to that class.
- A class can be considered as a user-defined data type, which defines the structure and behavior of objects that belong to that class.
- An object is an instance of a class.
- An *object* represents an entity in the real world that can be distinctly identified.
- For example, a class called “student" might have properties such as “fname," “lname," and “dept," as well as methods such as “getfname," “getdept," and “registration.“
- Once a class is defined, you can create one or more objects from it by using the "new" keyword.

Cont…

- An object has a unique identity, state, and behavior.
  - The *state* of an object (also known as its *properties* or *attributes*) is represented by *data fields* with their current values.
  - For Example:
    - A circle object has a data field **radius**, which is the property that characterizes a circle.
    - A rectangle object has the data fields **width** and **height**, which are the properties that characterize a rectangle.
  - The *behavior* of an object (also known as its *actions*) is defined by methods.
  - For example:
    - A circle object may invoke **getArea()** to return its area and **getPerimeter()** to return its perimeter and **setRadius(radius)** method to change its radius.

Cont…

- You can create many instances of a class.
- Creating an instance is referred to as *instantiation*.
- The terms *object* and *instance* are often interchangeable.
- The following diagram shows a class named **Circle** and its three objects.



Cont…

- Here's an example of a basic class and object definition in Java:

```
public class MyClass {
    private int myNumber; // instance variable declaration
    // constructor definition
    public MyClass(int number) {
        this.myNumber = number;
//this keyword commonly used within a class to reference instance variables
    }
    // method definition
    public void printNumber() {
        System.out.println("My number is " + this.myNumber);
    }}
Static void main(args[] string){
MyClass obj = new MyClass(42);
obj.printNumber(); // output: "My number is 42"
```

Defining Classes for Objects

- A Java class uses variables to define data fields and methods to define actions.
- A class has the following members:
  - Data fields/Attributes
  - Methods
  - Constructors

Java Class Attributes

- In Java, class attributes are also known as class variables or static variables.
- Class attributes are variables that are defined at the class level.
  - This means that the attribute is shared by all instances of the class.
  - You can access attributes by creating an object of the class, and by using the dot operator (.).
  - Or they can be accessed using the class name, followed by the dot operator and the name of the attribute.

Java Class Attributes

```
public class Person {
    static int count = 0; //  accessed without creating an instance of the class.
    String name; // accessed by creating an instance of the class.
    public Person(String name) {
        this.name = name;
        count++; // increment class attribute
    }
    public static void main(String[] args) {
        Person person1 = new Person("John");
        Person person2 = new Person("Jane");
       System.out.println("name:"+person1.name);
        System.out.println("Number of persons created: " + Person.count);
    }
}
```

Java Class Methods

- A method is a block of code that performs a specific task and can be called by other parts of a program.

Some common types of methods that can be defined in a Java class:

1. **Constructor method:** This method is used to create and initialize an object of a class. It has the same name as the class and does not have a return type.

2. **Getter and setter methods**: These methods are used to access and modify the values of private fields in a class.
    - Getter methods have a return type and return the value of a field, while setter methods do not have a return type and set the value of a field.

3. **Instance methods:** These methods are defined inside a class and can be called on an instance of that class. They can access the instance variables of the object they are called on.

4. **Static methods**: These methods are defined with the static keyword and can be called without creating an instance of the class. They cannot access instance variables, but they can access static variables.

Cont..

5. **Abstract methods:** These methods are declared without a body and are meant to be implemented by subclasses. They are used in abstract classes or interfaces.

6. **Final methods:** These methods cannot be overridden by subclasses.

7. **Private methods:** These methods are only accessible within the class they are defined in and cannot be called from outside the class.

8. **Protected methods:** These methods are accessible within the class they are defined in and in subclasses. They cannot be called from outside the class hierarchy.

Cont..

Example:

```
public class ExampleClass {
  // Declare a variable
  private int exampleVariable;
  // Constructor
  public ExampleClass(int initialVal) {
    this.exampleVariable = initialVal;
  }
  // A method that returns the value of the example variable
  public int getExampleVariable() {
    return exampleVariable;
  }
  // A method that sets the value of the example variable
  public void setExampleVariable(int newVal) {
    this.exampleVariable = newVal;
  }}
```

Java Class Constructor

- A constructor in Java is a special method that is used to initialize objects.
- The constructor is called when an object of a class is created. It is used to initialize the instance variables of the object and perform any other necessary setup.
- A constructor is invoked to create an object using the new operator.

They have **three** peculiarities:

- A constructor must have the same name as the class itself.
- Constructors do not have a return type, not even void.
- Constructors are invoked using the new operator when an object is created. Constructors play the role of initializing objects.

Types of constructors

- **Default Constructor**: If you do not define any constructor in your class, then a default constructor is provided by the Java compiler. It has no parameters and no implementation.
- **Parameterized Constructor:** A constructor that accepts one or more parameters is called a parameterized constructor. It is used to initialize the instance variables of the class.
- **Copy Constructor**: A copy constructor is used to create a new object with the same values as an existing object. It takes an object of the same class as a parameter and initializes the instance variables of the new object with the values of the existing object.

```
public class MyClass {
    private int num;

    // copy constructor
    public MyClass(MyClass other) {
        this.num = other.num;
    }
}
```

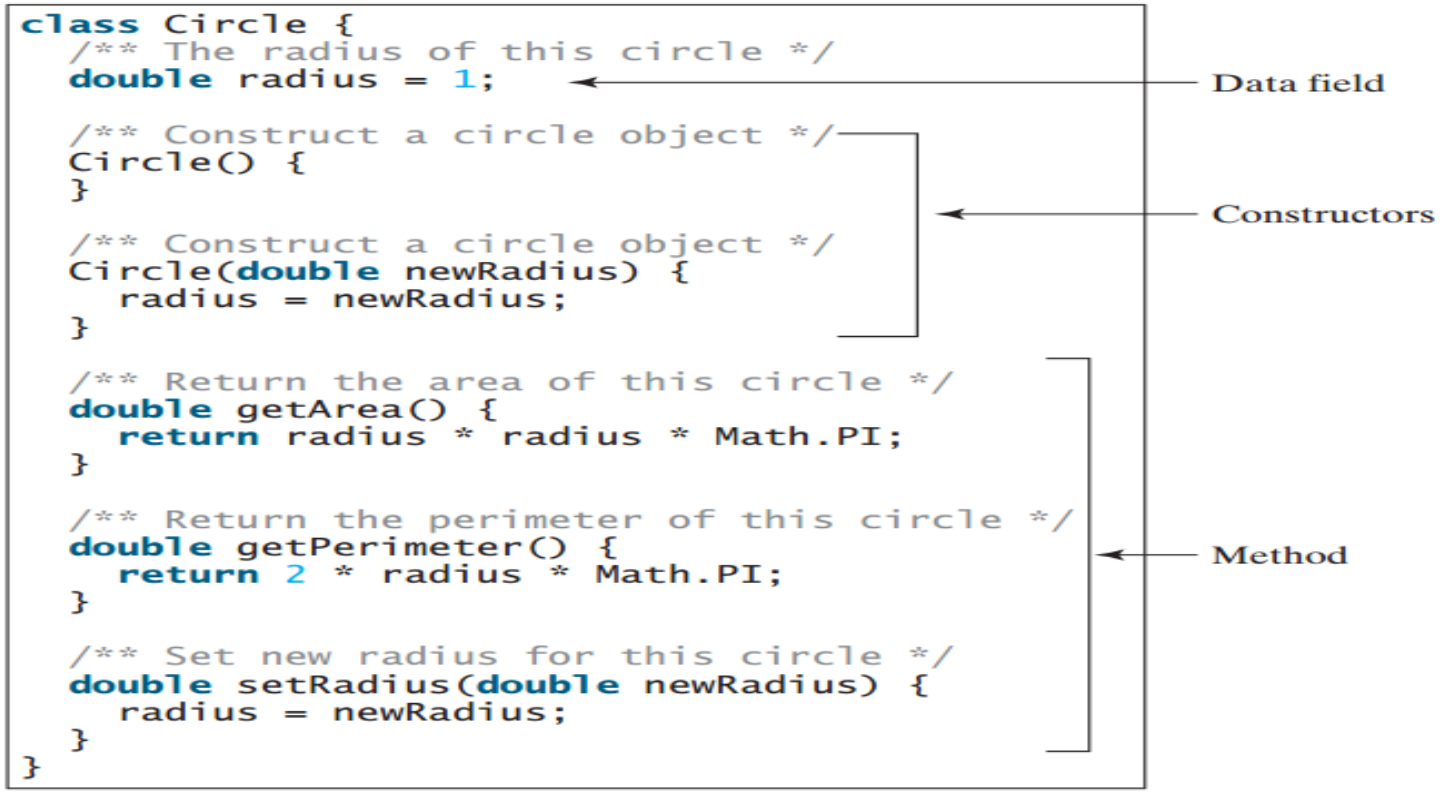
Example

```
public class Car {
    private String make;
    private String model;
    private int year;
    public Car(String make, String model, int year) {
        this.make = make;
        this.model = model;
        this.year = year;
    }}
```

- *In this example, the Car class has a constructor that takes three parameters: **make, model, and year**. When a new Car object is created, the constructor is called **automatically** with the specified arguments, and the instance variables make, model, and year are initialized with those values using the **this** keyword.*

Constructing Objects…

- Like regular methods, constructors can be overloaded (i.e., multiple constructors can have the same name but different signatures), making it easy to construct objects with different initial data values.
- Example of defining the class for circle objects.



Constructing Objects…

- To construct an object from a class, invoke a constructor of the class using the new operator, as follows:

  new ClassName(arguments);

- For example,

  Circle c1 = new Circle();

  Circle c2 = new Circle(25);

  Circle c3 = new Circle(125);

- A class normally provides a constructor without arguments (e.g., **Circle()**), such a constructor is referred to as a *no-arg* or *no-argument constructor*.
- A class may be defined without constructors.
- In this case, a public no-arg constructor with an empty body is implicitly defined in the class.
- This constructor, called a *default constructor*, is provided automatically *only if no constructors are explicitly defined in the class.*

Example: Defining Classes and Creating Objects

- *Classes are definitions for objects and objects are created from classes.*
- In this section we will consider two examples of defining classes and uses the classes to create objects.
- **Example 1:** Write a Java program that defines the **Circle** class and creates/constructs/ three circle objects with radius **1**, **25**, and **125** and displays the radius and area of each of the three circles.

Example: Defining…

- We can write this program in three ways:
  - 1st way: One class with one filename
  - 2nd way: Two class with one filename
  - 3rd way: Two class with different filename

1st Way Example: Defining… TestSimpleCircle.java

```
public class TestSimpleCircle{
  /** Main method */
  public static void main(String[] args){
  // Create a circle with radius 1
  TestSimpleCircle circle1 = new TestSimpleCircle();
  System.out.println("The area of the circle of radius"
  + circle1.radius + " is "+ circle1.getArea());
  // Create a circle with radius 25
   TestSimpleCircle circle2 = new TestSimpleCircle(25);
   System.out.println("The area of the circle of radius"
  + circle2.radius + " is " + circle2.getArea());
   // Create a circle with radius 125
   TestSimpleCircle circle3 = new TestSimpleCircle(125);
   System.out.println("The area of the circle of radius"
  + circle3.radius + " is " + circle3.getArea());
   // Modify circle radius
   circle2.radius = 100;
   System.out.println("The area of the circle of radius"
  + circle2.radius + " is " + circle2.getArea());
  }
double radius;
```

1st Way

Example: Defining…

```
  /** Construct a circle with radius 1 */
   TestSimpleCircle(){
   radius = 1;
   }
  /** Construct a circle with a specified radius */
   TestSimpleCircle(double newRadius) {
   radius = newRadius;
   }
   /** Return the area of this circle */
   double getArea(){
   return radius * radius * Math.PI;
   }
  /** Return the perimeter of this circle */
   double getPerimeter(){
   return 2 * radius * Math.PI;
   }
  /** Set a new radius for this circle */
   void setRadius(double newRadius){
   radius = newRadius;
   }
}
```

2st Way **Example: Defining…** TestSimpleCircle.java

```
public class TestSimpleCircle {
    /** Main method */
    public static void main(String[] args) {
    // Create a circle with radius 1
    SimpleCircle circle1 = new SimpleCircle();
    System.out.println("The area of the circle of radius" +
   circle1.radius + " is "+ circle1.getArea() );

   // Create a circle with radius 25
    SimpleCircle circle2 = new SimpleCircle(25);
    System.out.println("The area of the circle of radius" +
    circle2.radius + " is " + circle2.getArea());

    // Create a circle with radius 125
    SimpleCircle circle3 = new SimpleCircle(125);
    System.out.println("The area of the circle of radius " +
    circle3.radius + " is " + circle3.getArea());

    // Modify circle radius
    circle2.radius = 100; // or circle2.setRadius(100)
    System.out.println("The area of the circle of radius"
   +circle2.radius + " is "+ circle2.getArea());
    }
}
```

2st Way

Example: Defining…

```
// Define the circle class with two constructors
class SimpleCircle {
   double radius;

    /** Construct a circle with radius 1 */
    SimpleCircle(){
    radius = 1;
    }
    /** Construct a circle with a specified radius */
   SimpleCircle(double newRadius) {
    radius = newRadius;
    }
    /** Return the area of this circle */
   double getArea() {
    return radius * radius * Math.PI;
    }
    /** Return the perimeter of this circle */
    double getPerimeter(){
    return 2 * radius * Math.PI;
    }
    /** Set a new radius for this circle */
   void setRadius(double newRadius) {
    radius = newRadius;
    }
}
```

Example: Defining…

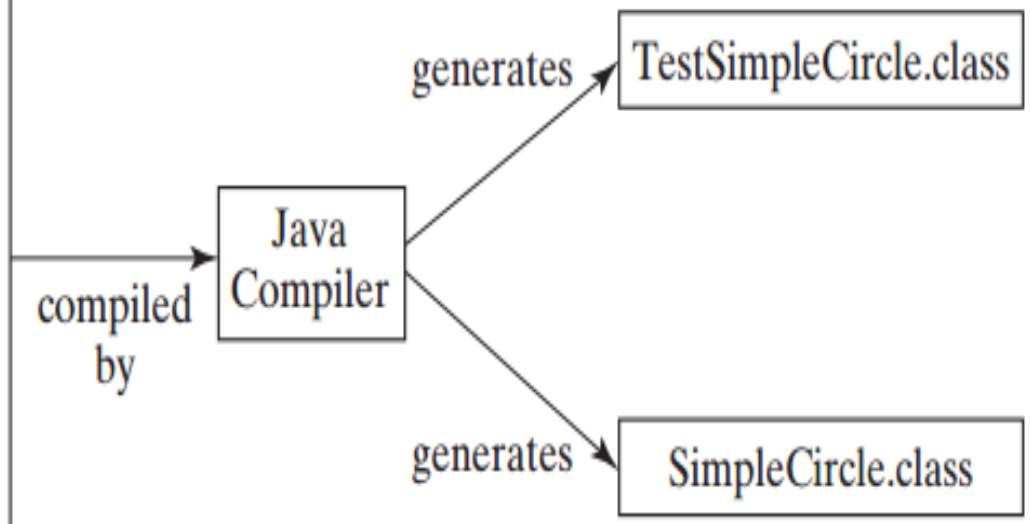
- This program contains two classes.
  - **TestSimpleCircle**, which is the main class.
  - **SimpleCircle ,** which is the client class
- When you run the program, the Java runtime system invokes the **main** method in the main class.
- You can put the two classes into one file, but only one class in the file can be a *public class*.
- Furthermore, the public class must have the same name as the file name.
- Therefore, the file name is **TestSimpleCircle.java**, since **TestSimpleCircle** is public.
- Each class in the source code is compiled into a .class file.
- When you compile **TestSimpleCircle.java,** two class files **TestSimpleCircle.class** and **SimpleCircle.class** are generated, as shown in the following Figure.

Example: Defining…

```
// File TestSimpleCircle.java

public class TestSimpleCircle {
  …
}

class SimpleCircle {
  …
}
```



3rd Way Example: Defining… TestSimpleCircle.java

```
public class TestSimpleCircle {
    /** Main method */
    public static void main(String[] args) {
    // Create a circle with radius 1
    SimpleCircle circle1 = new SimpleCircle();
    System.out.println("The area of the circle of radius" +
   circle1.radius + " is "+ circle1.getArea() );

   // Create a circle with radius 25
    SimpleCircle circle2 = new SimpleCircle(25);
    System.out.println("The area of the circle of radius" +
    circle2.radius + " is " + circle2.getArea());

    // Create a circle with radius 125
    SimpleCircle circle3 = new SimpleCircle(125);
    System.out.println("The area of the circle of radius " +
    circle3.radius + " is " + circle3.getArea());

    // Modify circle radius
    circle2.radius = 100; // or circle2.setRadius(100)
    System.out.println("The area of the circle of radius"
   +circle2.radius + " is "+ circle2.getArea());
    }
}
```

3rd way Example: Defining… SimpleCircle.java

```
// Define the circle class with two constructors
public class SimpleCircle {
   double radius;
    /** Construct a circle with radius 1 */
    SimpleCircle(){
    radius = 1;
    }
    /** Construct a circle with a specified radius */
   SimpleCircle(double newRadius) {
    radius = newRadius;
    }
    /** Return the area of this circle */
   double getArea() {
    return radius * radius * Math.PI;
    }
    /** Return the perimeter of this circle */
    double getPerimeter(){
    return 2 * radius * Math.PI;
    }
    /** Set a new radius for this circle */
   void setRadius(double newRadius) {
    radius = newRadius;
    }
}
```

Example: Defining…

- **Example 2:**
- Write a Java program that defines the **Rectangle** class and creates/constructs/ three rectangle objects with **height** and **width** data fields. The three rectangle objects should be entered from the keyboard using the scanner class in the main() method of your class and displays the area and perimeter of each of the three rectangles and finally show the UML class diagram.

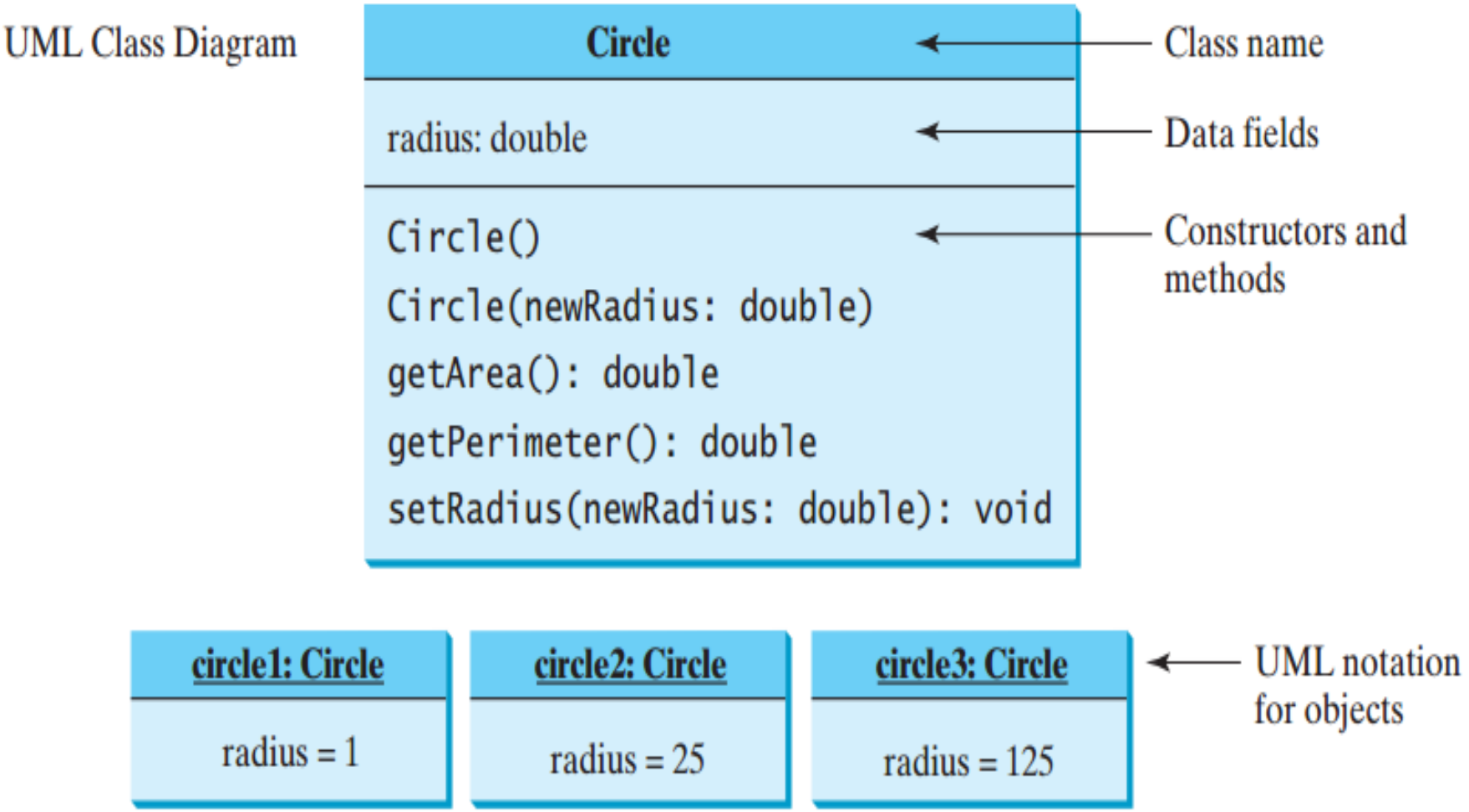
Unified Modeling Language (UML) **notation.**

- Unified Modeling Language (UML) is the most widely used graphical scheme for modeling object-oriented systems.
- UML is used to illustrate class templates, objects, data fields and methods in a class.
- In the class diagram, the data field is denoted as dataFieldName: dataFieldType
- The constructor is denoted as

  ClassName(parameterName: parameterType)
- The method is denoted as

  methodName(parameterName:parameterType): returnType

Unified Modeling…



Accessing Objects via Reference Variables

- *An object's data and methods can be accessed through the dot (.) operator via the object's reference variable.*

1. Reference Variables and Reference Types

- Objects are accessed via the object's *reference variables*, which contain references to the objects.
- A class is a *reference type*, which means that a variable of the class type can reference an instance of the class.
- Example:

```
SimpleCircle circle1 = new SimpleCircle();
```

Accessing Objects…

2. Accessing an Object's Data and Methods

- After an object is created, its data can be accessed and its methods invoked using the *dot operator*(**.**), also known as the *object member access operator*:
- **objectRefVar.dataField** references a data field in the object.
- **objectRefVar.method(arguments)** invokes a method on the object.
- For example, **circle1.radius** references the radius in **circle1**, and **circle1.getArea()** invokes the **getArea** method on **circle1**.

Accessing Objects…

3. Reference Data Fields and the *null* Value

- If a data field of a reference type does not reference any object, the data field holds a special Java value, **null** .
- For example, the following **Student** class contains a data field **name** of the **String** type.
- **String** is a predefined Java class.

```
Public class Student {
  String name; // name has the default
  value null
  int age; // age has the default value 0
  boolean isScienceMajor;//isScienceMajor
  has default value false
  char gender; // gender has default
  value '\u0000‘ or white space
}
```

Accessing Objects…

- If a data field of a reference type does not reference any object, the data field holds a special Java value, **null** .
- The default value of a data field is **null** for a reference type, **0** for a numeric type, **false** for a **boolean** type, and **\u0000** for a **char** type.
- However, Java assigns no default value to a local variable inside a method.
- The following code displays the default values of the data fields **name**, **age**, **isScienceMajor**, and **gender** for a **Student** object:

Accessing Objects…

```
Public class TestStudent{
  public static void main(String[]
  args){
  Student student = new Student();
  System.out.println("name? " +
  student.name );
  System.out.println("age? " +
  student.age);
  System.out.println("isScienceMajor?
  " + student.isScienceMajor);
  System.out.println("gender? " +
  student.gender);
  }
}
```

Accessing Objects…

- The following code has a compile error, because the local variables **x** and **y** are not initialized:

```
Public class TestStudent{
  public static void main(String[]
  args){
  int x; // x has no default value
  String y;//y has no default value
  System.out.println("x is " + x);
  System.out.println("y is " + y);
  }
}
```

Static Variables, Constants, and Methods

- In Java, a static variable is a variable that belongs to the class itself, rather than any particular instance of the class.
- *A static variable is a variable that is shared among all instances of a class. It is not tied to any particular instance of the class, but rather to the class itself.*
- Static variables are declared using the static keyword and can be accessed directly using the class name.

```
public class ExampleClass {
  static int count = 0; // static variable
  public ExampleClass() {
    count++; // increment the count each time a new instance is created
  }
  public static void main(String[] args) {
    ExampleClass obj1 = new ExampleClass();
    ExampleClass obj2 = new ExampleClass();
    ExampleClass obj3 = new ExampleClass();
    System.out.println("Number of instances created: " + count); // accessing the static variable
  }}
```

Static Variables, Constants, and Methods

- A constant variable is a variable that cannot be changed once it has been assigned a value.
- Constants are typically used for values that are fixed and do not change during the execution of the program.
- In Java, a variable declared with the final keyword becomes a constant variable, which means its value cannot be changed once it has been initialized.

```
public class Example {
  public static void main(String[] args) {
    final int MAX_VALUE = 100; // declaring a constant variable
    // MAX_VALUE = 200; // this will result in a compile-time error
    System.out.println("The maximum value is " + MAX_VALUE);
  }
}
// MAX_VALUE is declared as a constant variable with the final keyword, and its value is set to 100. Trying to change the value of MAX_VALUE will result in a compile-time error.
```

Static Variables, Constants, and Methods

- In Java, **a static method** is a method that belongs to the class rather than an instance of the class. This means that you can call a static method without creating an instance of the class.

Some key points about static methods in Java

- Static methods are declared using the keyword "static" before the return type of the method.
- You can call a static method using the class name followed by the method name, without creating an object of the class.
- Static methods cannot access instance variables or instance methods of the class. They can only access static variables and other static methods.
- You cannot override a static method. When you declare a static method in a subclass with the same name and signature as a static method in the superclass.
- The main() method in Java is a static method that is called by the JVM to start the execution of a Java program.

Static Variables, Constants, and Methods

Static method Example

```
public class Calculator {
    public static int add(int a, int b) {
        return a + b;
    }
}
// To call the static method, you do not need to create an instance of Calculator class.
int result = Calculator.add(5, 10); // or int result=add(5,10);
System.out.println(result); // Output: 15
```

- In this example, the Calculator class has a static method named add(), which takes two integer arguments and returns their sum. This method is declared using the static keyword, which means that you can call it without creating an instance of the Calculator class.
- To call the add() method, you simply use the class name Calculator followed by the method name add() and pass in the two integer arguments. The return value of the method is stored in the result variable and printed to the console.

instance vs static methods in Java

- Instance methods are called on an instance of a class, while static methods are called on the class itself.
- Instance methods can access and modify instance variables and instance methods of the class, while static methods can only access and modify static variables and static methods of the class.
- To call an instance method, you need an instance of the class, while to call a static method, you don't need an instance of the class.

Java Access Modifiers

- *In Java, an access modifier is a keyword that determines the accessibility of a class, interface, method, or variable from other parts of the program.*
- There are four access modifiers in Java:

1. public: makes a class, method, or variable accessible to all other classes and packages.

2. private: restricts the accessibility of a class, method, or variable to only the same class it is defined in.

3. protected: allows the class, method, or variable to be accessed within the same package and subclasses .

4. default: If no access modifier is specified, then the class, method, or variable has default access, which restricts its accessibility to only the same package.

Public Access modifier

- Public access modifier allows the member variables and methods of a class to be accessible from any other part of the program.
- Example:

```
public class Person {
   public String name;
   public int age;
   public void sayHello() {
System.out.println("Hello, my name is " + name + " and I am " + age + " years old.");
   }}
```

- In this example, the Person class has two public member variables name and age, and a public method sayHello().
- These members can be accessed from any other part of the program by creating an instance of the Person class and calling the public members on that instance.

```
Person john = new Person();
john.name = "John";
john.age = 30;
john.sayHello(); // Output: "Hello, my name is John and I am 30 years old."
```

Private Access modifier

- In Java, the private modifier is used to restrict access to a member of a class to only the class itself. Example:

```
public class MyClass {
    private int x;
    public MyClass(int x) {
        this.x = x;
    }
    private void foo() {
        System.out.println("x is " + x);
    }
    public void bar() {
        foo(); // Can access the private method foo() from within the class
    }}
public class Main {
    public static void main(String[] args) {
        MyClass obj = new MyClass(10);
        obj.bar(); // Output: x is 10
        // obj.foo(); // Error: foo() has private access in MyClass
        // System.out.println(obj.x); // Error: x has private access in MyClass
    }}
```

Protected Access modifier

- In Java, the protected modifier is used to declare a member of a class that can be accessed within the same package or by subclasses in any package. Example:

```
package com.example;
public class Animal {
   protected String name;
   protected void eat() {
      System.out.println(name + " is
eating.");
   }}
```

```
package com.example;
public class Main {
public static void main(String[] args) {
      Dog dog = new Dog("Bob");
      dog.bark();
 // Output: Bob is barking.
      dog.feed();
// Output: Bob is eating.
   }}
```

```
package com.example;
public class Dog extends Animal {
   public Dog(String name) {
      this.name = name;
   }
   public void bark() {
      System.out.println(name + " is barking.");
   }
   public void feed() {
      eat(); // Can access the protected method
eat() from the superclass Animal
   }}
```

Default Access modifier

- In Java, if a class member is not explicitly declared with any access modifier (public, private, or protected), then it has default access. This means that the member is accessible within the same package but not outside of it. Example:

```
package com.example;
class Animal {
   String name;
   void eat() {
      System.out.println(name + " is eating.");
   }}
public class Main {
   public static void main(String[] args) {
      Animal animal = new Animal();
      animal.name = "Buddy"; // Can access the package-private field name from the same package
      animal.eat(); // Can access the package-private method eat() from the same package
   }}
```

Visibility Modifiers

```
package p1;

public class C1 {
  public int x;
  int y;
  private int z;

  public void m1() {
  }
  void m2() {
  }
  private void m3() {
  }
}
```

```
package p1;

public class C2 {
  void aMethod() {
    C1 o = new C1();
    can access o.x;
    can access o.y;
    cannot access o.z;

    can invoke o.m1();
    can invoke o.m2();
    cannot invoke o.m3();
  }
}
```

```
package p2;

public class C3 {
  void aMethod() {
    C1 o = new C1();
    can access o.x;
    cannot access o.y;
    cannot access o.z;

    can invoke o.m1();
    cannot invoke o.m2();
    cannot invoke o.m3();
  }
}
```

Visibility Modifiers

- If a class is not defined as public, it can be accessed only within the same package.

```
package p1;

class C1 {
  ...
}
```

```
package p1;

public class C2 {
  can access C1
}
```

```
package p2;

public class C3 {
  cannot access C1;
  can access C2;
}
```

Data Field Encapsulation in java

- *Data field encapsulation is the process of hiding the internal state of an object and providing controlled access to that state through well-defined methods.*
- *Encapsulation is a fundamental principle of object-oriented programming and helps to ensure that the state of an object is accessed and modified only through the appropriate methods.*
- *In object-oriented programming, encapsulation is achieved using access modifiers and getter and setter methods.*
- A private data field cannot be accessed by an object from outside the class that defines the private field.
- However, a client often needs to retrieve and modify a data field.
- To make a private data field accessible, provide a *get* method to return its value.
- To enable a private data field to be updated, provide a *set* method to set a new value.

Data Field Encapsulation

- For example, consider the following class that represents a simple person:

```java
public class Person {
    private String name;
    private int age;

    public String getName() {
        return name;
    }

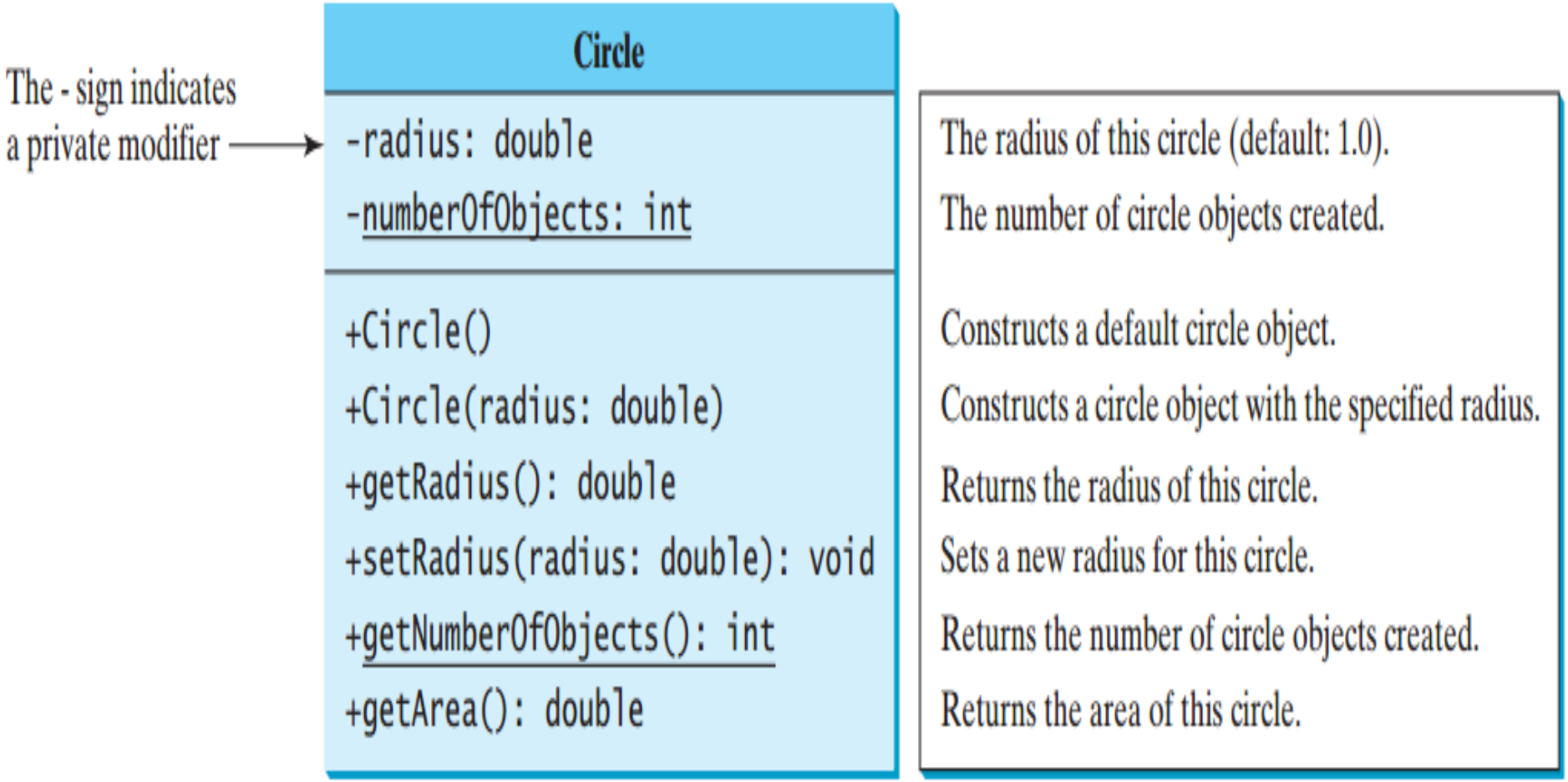
    public void setName(String name) {
        this.name = name;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        this.age = age;
    }
}
```

- In this example, the name and age fields are marked as private, which means that they cannot be accessed directly from outside the Person class. Instead, the class provides public getter and setter methods that allow external code to retrieve or modify the values of these fields in a controlled manner.
- By encapsulating the name and age fields, we ensure that their values can only be modified through the appropriate methods.

Data Field Encapsulation



Data Field Encapsulation

```
public class CircleWithPrivateDataFields {
    /** The radius of the circle */
  private double radius = 1;
  /** The number of objects created */
  private static int numberOfObjects = 0;
  /** Construct a circle with radius 1 */
   public CircleWithPrivateDataFields() {
   numberOfObjects++;
   }
  /** Construct a circle with a specified radius */
   public CircleWithPrivateDataFields(double
  newRadius) {
   radius = newRadius;
   numberOfObjects++;
   }
   /** Return radius */
  public double getRadius() {
   return radius;
   }
```

Data Field Encapsulation

```
  /** Set a new radius */
   public void setRadius(double newRadius)
   {
   radius=(newRadius>=0)? newRadius: 0;
   }
   /** Return numberOfObjects */
   public static int getNumberOfObjects()
   {
    return numberOfObjects;
   }
    /** Return the area of this circle */
    public double getArea() {
    return radius * radius * Math.PI;
    }
}
```

Data Field Encapsulation

```
public class TestCircleWithPrivateDataFields {
    /** Main method */
    public static void main(String[] args) {
    // Create a circle with radius 5.0
    CircleWithPrivateDataFields myCircle =
    new CircleWithPrivateDataFields(5.0);
    System.out.println("The area of the circle of
  radius " + myCircle.getRadius() + " is "+
  myCircle.getArea());
    // Increase myCircle's radius by 10%
    myCircle.setRadius(myCircle.getRadius() * 1.1);
    System.out.println("The area of the circle of
  radius " + myCircle.getRadius() + " is "+
  myCircle.getArea());
    System.out.println("The number of objects
  created is "+
  CircleWithPrivateDataFields.getNumberOfObjects())
  ;
    }
}
```

Data Field Encapsulation

- The data field **radius** is declared private.
- Private data can be accessed only within their defining class, so you cannot use **myCircle.radius** in the client program.
- A compile error would occur if you attempted to access private data from a client.
- Since **numberOfObjects** is private, it cannot be modified.
- This prevents changing of data without authority.

this Keyword in Java

- In Java, "this" is a keyword that refers to the current object.
- It can be used to refer to the instance variables or methods of the current object.

common uses of the "this" keyword in Java

1. To refer to instance variables: If a method parameter or local variable has the same name as an instance variable, you can use "this" to refer to the instance variable.

```
public class MyClass {
  private int x;
  public void setX(int x) {
    this.x = x;
  }}
```

In this example, "this.x" refers to the instance variable "x", while "x" refers to the method parameter.

Cont…

common uses of the "this" keyword in Java

2. To call another constructor in the same class: If a class has multiple constructors, one constructor can call another constructor using the "this" keyword. For example:.

```
public class MyClass {
  private int x;
  public MyClass() {
    this(0); // calls the constructor with one int parameter
  }
  public MyClass(int x) {
    this.x = x;
  }}
```

- In this example, the first constructor calls the second constructor using "this(0)".

Cont…

common uses of the "this" keyword in Java

3. To return the current object: In Java, the **this keyword** is used to refer to the current object instance. You can use the this keyword to return the current object in a method. For example:

```
public class MyClass {
  private int value;
  public MyClass(int value) {
    this.value = value;
  }
  public MyClass getObject() {
    return this; // returns the current object instance
  }
   public static void main(String[] args)
   {
      MyClass obj=new MyClass(10);
      System.out.print("Current Object="+obj.getObject());
   }} //output: Current Object=MyClass
```

- In the getObject() method, this is used to return the current object instance of the MyClass class.

Reading Assignment: *Passing Objects to Methods*

```
class Data {

    int data1;
    int data2;
}

class SetData {

    void setData(Data da,int d1,int d2)
    {
        da.data1 = d1;
        da.data2 = d2;
    }

    void getData(Data da)
    {
        System.out.println("data1 : "+da.data1);
        System.out.println("data2 : "+da.data2);
    }
}

public class Javaapp {

    public static void main(String[] args) {

        Data da = new Data();
        SetData sd = new SetData();
        sd.setData(da,50,100);
        sd.getData(da);
    }
}
```

Reading Assignment: *Array of Objects*

```
public class Person_ex {
    private int age;
    private String name;
    private char gender;

    public Person_ex(int age1, String name1, char gender1) {
        age = age1;
        name = name1;
        gender = gender1;
    }

    public int getAge() {
        return age;
    }

    public String getName() {
        return name;
    }

    public char getGender() {
        return gender;
    }

    public static void main(String[] args) {
        Person_ex[] p = new Person_ex[3];
        p[0] = new Person_ex(20, "Ahsan", 'M');
        p[1] = new Person_ex(25, "Ayesha", 'F');
        p[2] = new Person_ex(30, "Akbar", 'M');

        for (int i = 0; i < p.length; i++) {
            System.out.println(p[i].getName() + " " + p[i].getAge() + " "
                    + p[i].getGender());
        }
    }
}
```

The End!!